

2048 Master How-To Guide

CMPT 310, Group 1

github.com/praneetkb/2048-Master

Introduction

2048 Master is a project that builds an AI agent to play 2048, the sliding-tile puzzle game. The game is played on a 4×4 grid of tiles. Each turn, the agent slides all tiles in one of four directions (up, down, left, right). When two tiles with the same value collide, they merge into a single tile with twice the value, and the merged tile's value is added to the score. A move is legal if applying it changes the board. After every legal move, a random tile with value 2 or 4 is spawned. The game ends when no legal move remains.

2048 is a reinforcement learning problem: the agent receives the board state as input and produces one of the four moves as output. Random tile spawning makes the environment stochastic, so the same move from the same board can lead to different results. The reward is the score gained from merges, and the agent's goal is to maximize the score it can reach over a game.

This guide explains how we built the agent. We first implement the game environment, then build an N-tuple network and train it with temporal difference (TD) learning through self-play. We use the learned value function as the evaluator of an Expectimax search and compare the final agent against a random baseline. The guide also covers the Pygame interface and common troubleshooting issues.

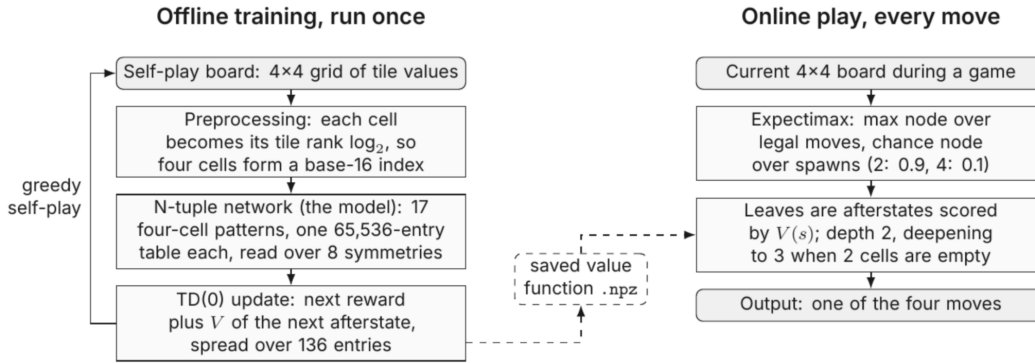


Figure 1. The 2048 Master reinforcement learning pipeline.

Setting Up the 2048 Environment

1. Board Representation

We represent the 2048 board as a 4×4 NumPy array. Each cell stores the numerical value of the tile at that position, while an empty cell is represented by 0:

`[0, 2, 0, 0] [0, 0, 0, 0] [0, 0, 4, 0] [2, 0, 0, 0]`

This representation allows the environment to perform board operations efficiently and provides the current game state directly to the AI agents.

2. Movement and Merging

A single move consists of sliding all tiles toward the selected direction and merging adjacent tiles with the same value. For a left move, we remove empty cells, merge equal adjacent tiles, then slide the row again:

Before: `[2, 2, 4, 0]` **Merge:** `[4, 0, 4, 0]` **Slide:** `[4, 4, 0, 0]`

Rather than implementing four independent movement algorithms, we reuse the same left-movement logic through board transformations:

- Left: apply the left-movement logic directly.
- Right: reverse each row, apply left movement, then reverse it back.
- Up: transpose the board, apply left movement, then transpose it back.
- Down: transpose the board, apply right movement, then transpose it back.

This reduces duplicated code and keeps the merging behavior consistent across all directions.

```
# Performs a full LEFT move on a single row (slide + merge + slide)
def move_row_left(row):
    # Step 1: slide
    slid_row = slide_row_left(row)

    # Step 2: merge
    merged_row, merged_values, merged_changed = merge_row_left(slid_row)

    # Step 3: slide again (to remove gaps after merge)
    final_row = slide_row_left(merged_row)

    # Check if any changes occurred (either from sliding or merging)
    changed = not np.array_equal(row, final_row) or merged_changed

    return final_row, merged_values, changed

# Performs a full LEFT move on the entire board
# It applies row-based movement to every row
def move_left(board):
    new_board = np.zeros_like(board)
    all_merged_values = []
    changed = False

    for i in range(board.shape[0]):
        new_row, merged_values, row_changed = move_row_left(board[i])

        new_board[i] = new_row
        all_merged_values.extend(merged_values)

        if row_changed:
            changed = True

    return new_board, all_merged_values, changed
```

Figure 2. Code excerpt implementing left movement and merging.

3. Random Tile Spawning

At the beginning of a game, two tiles are placed on randomly selected empty cells. After each legal move, the environment randomly selects an empty cell and places either a 2, with 90% probability, or a 4, with 10% probability. The movement functions return a `changed` flag, so a new tile is spawned only when the move modified the board. This makes 2048 a stochastic environment: the agent controls its actions, but it cannot control where or which value the next tile will appear.

4. Rewards

The environment uses the score increase from tile merges as the immediate reward. When two equal tiles merge, the resulting tile’s value is added to the score. For example, $[4, 4] \rightarrow [8]$ gives a reward of 8. A move that only rearranges tiles receives a reward of 0. If multiple merges occur, their values are added to produce the total reward for that transition.

Building the N-Tuple Value Function

The RL agent needs a value function $V(s)$ that estimates how good a board position is. We represent it with an N-tuple network, a set of lookup tables that evaluate a board by summing selected entries. An N-tuple network captures useful local board structure with compact tables that can be trained efficiently using TD learning.

Tile ranks. Every tile is a power of two, so instead of storing the value itself, we store its exponent. An empty cell becomes 0, and the tiles 2, 4 and 8 become 1, 2 and 3. This gives each cell one of 16 possible symbols.

Tuple patterns. A tuple is a fixed pattern of four cells. We use 17 tuples: the four rows, the four columns, and the nine overlapping 2×2 squares. These patterns capture the local interactions used to evaluate a board.

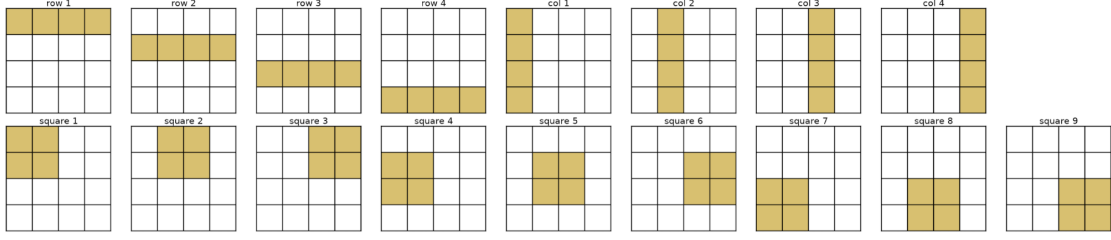


Figure 3. The 17 four-cell tuple patterns.

Lookup tables. Each tuple pattern has its own lookup table, with one slot for every possible combination of values its four cells can contain. With 16 possible values per cell, each table contains $16^4 = 65,536$ slots. The network reads the slot selected by each pattern and sums the values it finds. Initially, all slots are zero.

Base-16 encoding. The four values in each tuple are converted into a single table index. The first cell contributes the units digit, while the remaining cells contribute multiples of 16, 256 and 4096. The tiles $[8, 4, 2, 2]$ have exponents $[3, 2, 1, 1]$, which encode as $3 + 2(16) + 1(256) + 1(4096) = 4387$.

```
def encode(tile_values):
    ranks = tile_ranks(tile_values)          # tiles -> exponents, e.g.
    [8,4,2,2] -> [3,2,1,1]
    return ranks[0] + 16 * ranks[1] + 256 * ranks[2] + 4096 * ranks[3]
```

Figure 4. Base-16 tuple encoding in the N-tuple network.

Symmetry handling. Rotated and reflected versions of the same board are strategically equivalent. Each board has eight orientations: four rotations and four rotations of a reflected version. We evaluate each board across all eight orientations using the same lookup tables. With 17 patterns and 8 orientations, each board contributes to 136 table entries. During training, the TD correction is divided evenly across these entries. We use `np.add.at` so entries selected more than once are updated correctly.

```
def value(self, grid: np.ndarray, validate: bool = True) -> float:
    indices = self.feature_indices(grid, validate=validate)
    table_numbers = np.arange(len(self.tuples), dtype=np.intp)[None, :]
    return float(self.tables[table_numbers,
                             indices].sum(dtype=np.float64))

def update(self, grid: np.ndarray, delta: float) -> None:
    """Distribute delta evenly over every table lookup used by value()."""
    try:
        numeric_delta = float(delta)
    except (TypeError, ValueError) as error:
        raise TypeError("delta must be a finite real number") from error
    if not np.isfinite(numeric_delta):
        raise ValueError("delta must be finite")

    indices = self.feature_indices(grid, validate=False)
    amount_per_lookup = numeric_delta / indices.size
    offsets = np.arange(len(self.tuples), dtype=np.int64) * TABLE_SIZE
    np.add.at(self.tables.reshape(-1), (indices + offsets).reshape(-1),
              amount_per_lookup)
```

Figure 5. Symmetry-aware evaluation and duplicate-safe updates.

Training with TD Learning

We train the N-tuple value function through self-play. Its lookup tables are initially zero. As the agent plays games, TD updates adjust these values based on the rewards and future afterstates it encounters.

1. Afterstates

An afterstate is the board immediately after the player’s move has been applied, but before the random tile is spawned. A move in 2048 is deterministic, so afterstates represent the direct result of the agent’s decision before randomness enters. This lets the agent evaluate the resulting board for each legal action directly.

2. TD Update

For each possible action, the trainer evaluates the resulting afterstate by combining the immediate merge reward with the learned value of that afterstate. The action with the highest value is selected:

$$\begin{aligned}Q(s, a) &= r(s, a) + V(s'_a), \\ \text{target} &= r_{t+1} + V(s'_{t+1}), \\ \delta_t &= \text{target} - V(s'_t), \\ V(s'_t) &\leftarrow V(s'_t) + \alpha \delta_t.\end{aligned}$$

The value function estimates future rewards from an afterstate, so the immediate reward is handled separately during action selection. A terminal state has no future reward, so its TD target is zero.

3. Self-Play Training Loop

The agent generates training data by repeatedly playing complete games of 2048. For each episode:

1. Start a new board and spawn the initial two tiles.
2. Test the four moves and discard actions that do not change the board.
3. Apply each legal action without spawning a tile, then calculate its reward plus N-tuple value.
4. Choose the action with the highest estimated value, execute it, and spawn a random tile.
5. Calculate the TD target and error, then update the previous afterstate's N-tuple values.
6. Continue until no legal moves remain, then start the next episode.

```
next_action, next_afterstate, _next_reward, target = self.evaluate_moves(
    game.board.grid
)
if next_action is None: # terminal: no future reward
    target = 0.0

delta = target - self.network.value(afterstate, validate=False)
self.network.update(afterstate, self.alpha * delta)
```

Figure 6. The main TD self-play update.

4. Training Configuration

We trained the agent for 10,000 episodes to expose the value function to a wide range of board configurations. We experimented with different learning rates and selected the final value based on training performance and stability. Checkpoints are saved as `.npz` files containing the current N-tuple lookup tables, allowing training to resume without starting from zero.

```
class TDTrainer:
    def __init__(
        self,
        network: NTupleNetwork,
        learning_rate=0.01,
        episodes=10000, # 10,000 games
        log_every=500, # print + checkpoint interval
        checkpoint_path="checkpoints/value_function.npz",
        log_path="training_logs/training_scores.csv",
    ):
        pass
```

Figure 7. The training configuration excerpt.

5. Monitoring Training Progress

We record each episode's score and calculate the average score. An increasing average indicates that the agent is generally learning to produce better game outcomes over time.

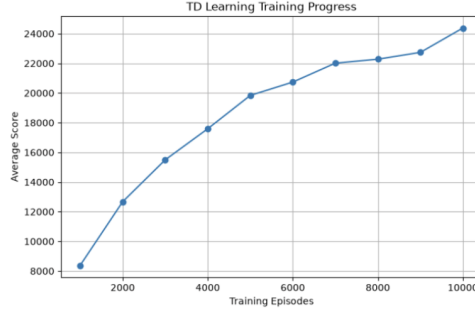


Figure 8. Average score during 10,000 self-play episodes.

Integrating the Learned Value Function with Expectimax

Expectimax needs a model of the game’s randomness and a way to evaluate board positions. Chance nodes model the random spawning of a 2 or 4 tile. Our original Expectimax agent used a hand-designed heuristic. For the RL agent, we replaced that heuristic with the value function learned through TD self-play.

The search alternates between max nodes, where the agent chooses the best legal move, and chance nodes. We used a search depth of 2:

max node → chance node → max node → chance node → evaluation function

At the leaf nodes, the N-tuple network evaluates the resulting board. Expectimax then determines which initial move has the highest expected outcome.

```
# Learned value function or the previous heuristic can be used.
def evaluate(self, state):
    if self.network is not None:
        return self.network.value(state, validate=False)
    empty = len(empty_cells(state))
    max_tile = np.max(state)
    return (100 * empty) + max_tile
```

Figure 9. The evaluation function used at Expectimax leaf nodes.

Evaluating the Agent

We compared the Random agent, heuristic Expectimax, and Expectimax + RL in the same environment. The two Expectimax agents use the same depth-2 search; only their leaf evaluation differs. Each agent played 30 games.

Agent	Games	Avg	Median	Best	Best tile	Reached 2048	ms/move
Random	30	1,178	990	3,076	256	0/30	0.0
Expectimax (heuristic)	30	14,724	15,698	27,308	2,048	5/30	8.8
Expectimax + RL	30	62,244	66,646	120,744	8,192	30/30	12.9

Table 1. Results from 30 games per agent.

The RL agent achieved an average score more than four times higher than heuristic Expectimax and reached 2048 in all 30 games. Its best game reached 8192 with a score of 120,744. The RL agent takes about 4 ms more per move, which remains fast enough for real-time gameplay. Since both Expectimax agents use the same search, the difference comes from the learned N-tuple value function.

UI: The Pygame Interface

The UI is separated from the game logic. The renderer displays the game, the environment handles the rules and state, and the AI agents independently select actions. The main menu offers the three agents and the comparison screen, selected with keys 1 to 4 or by clicking the buttons.

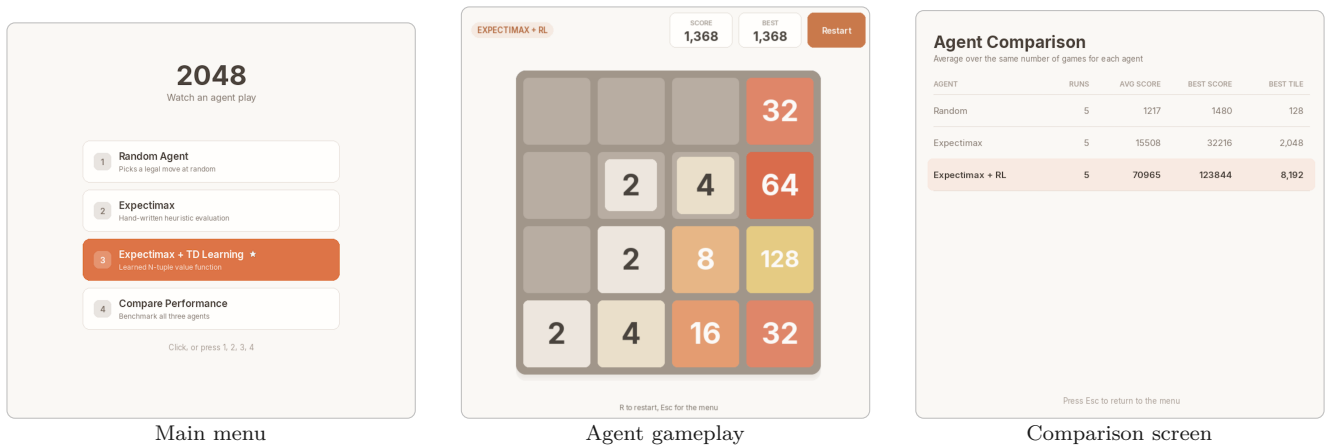


Figure 10. The Pygame interface.

Picking an agent starts a game and the agent plays by itself. Tiles slide and merge with animation, and the score updates in the header. Press **R** to restart, or **Esc** to return to the menu. When the game ends, an overlay shows the final score and best tile. The comparison screen runs five games per agent and reports the average score, best score and highest tile.

Troubleshooting

Problem 1: `ModuleNotFoundError: No module named 'agents'`

Running `python3 training/td_learning.py` directly can fail because Python does not treat the project root as the package root. From the project root, run:

```
python3 -m training.td_learning
```

Problem 2: `ModuleNotFoundError: No module named 'pygame'`

This means the current Python environment does not have the required dependencies. From the project root, run:

```
pip install -r requirements.txt    python3 main.py
```

Conclusion

We built the environment, trained an N-tuple value function through TD self-play, and used it in Expectimax to replace the hand-designed heuristic.

The authors retain full copyright of this guide and grant Simon Fraser University (SFU) permission to publish and share it for academic and educational purposes, including use by future students.